

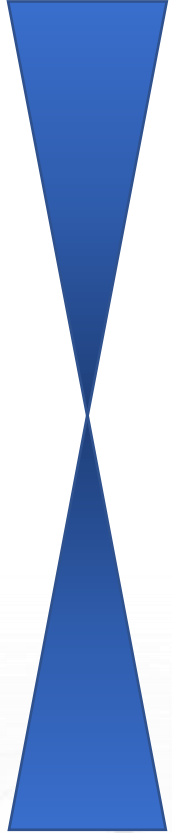


Modeling

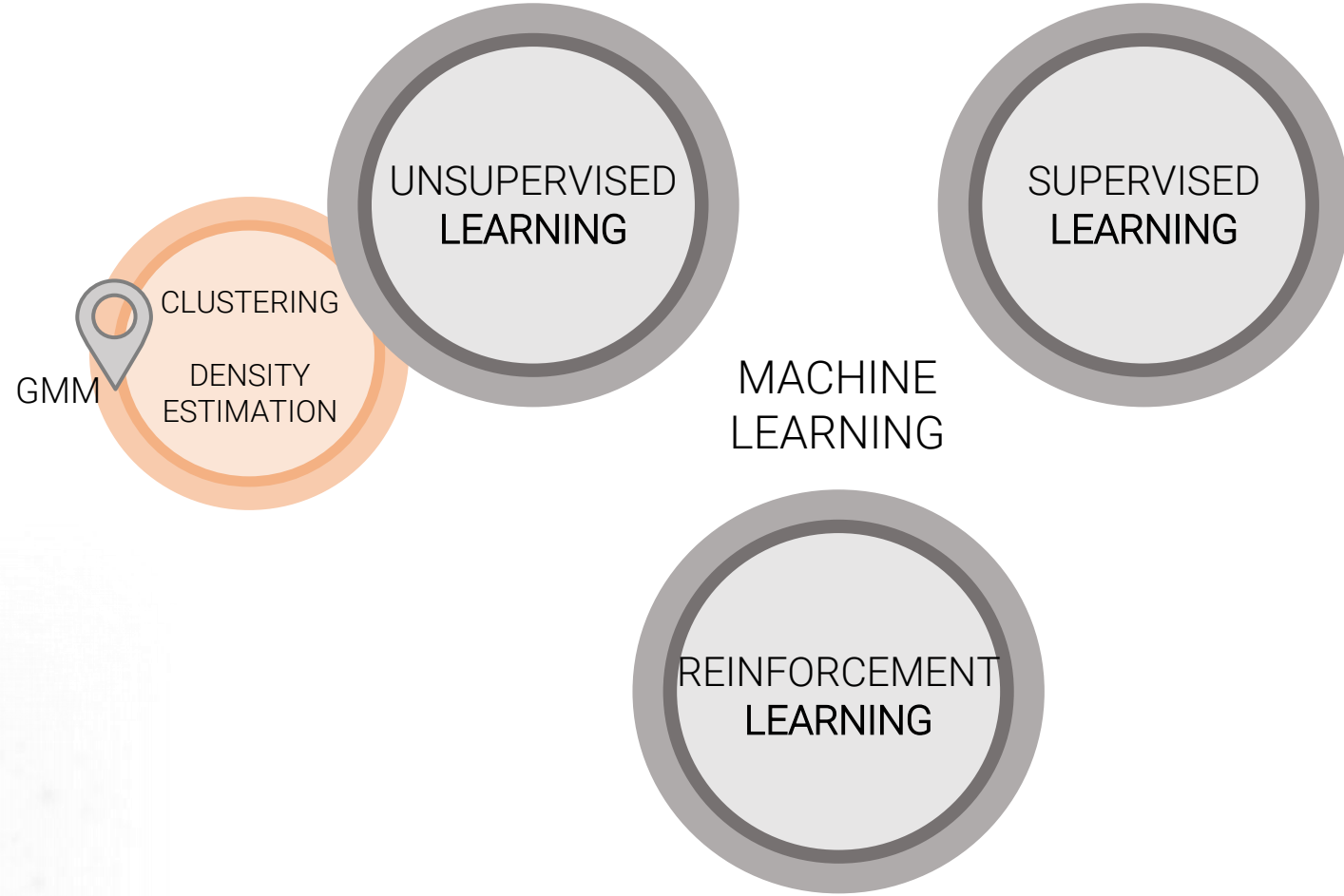
Unsupervised Learning: Gaussian Mixture Models

Agenda

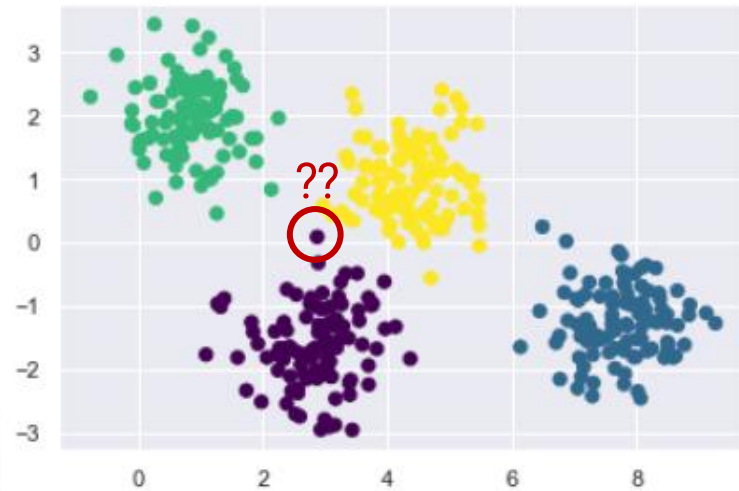
- Motivation von GMMs
- GMMs und Maximum Likelihood Estimation
- GMM und Informationskriterien



Gaussian Mixture Models: wo befinden wir uns?



Ausgangslage: beyond k -means & DBSCAN



Was denken Sie?

Hatten wir schon eine Möglichkeit kennengelernt, die uns in die angestrebte Richtung bringt?

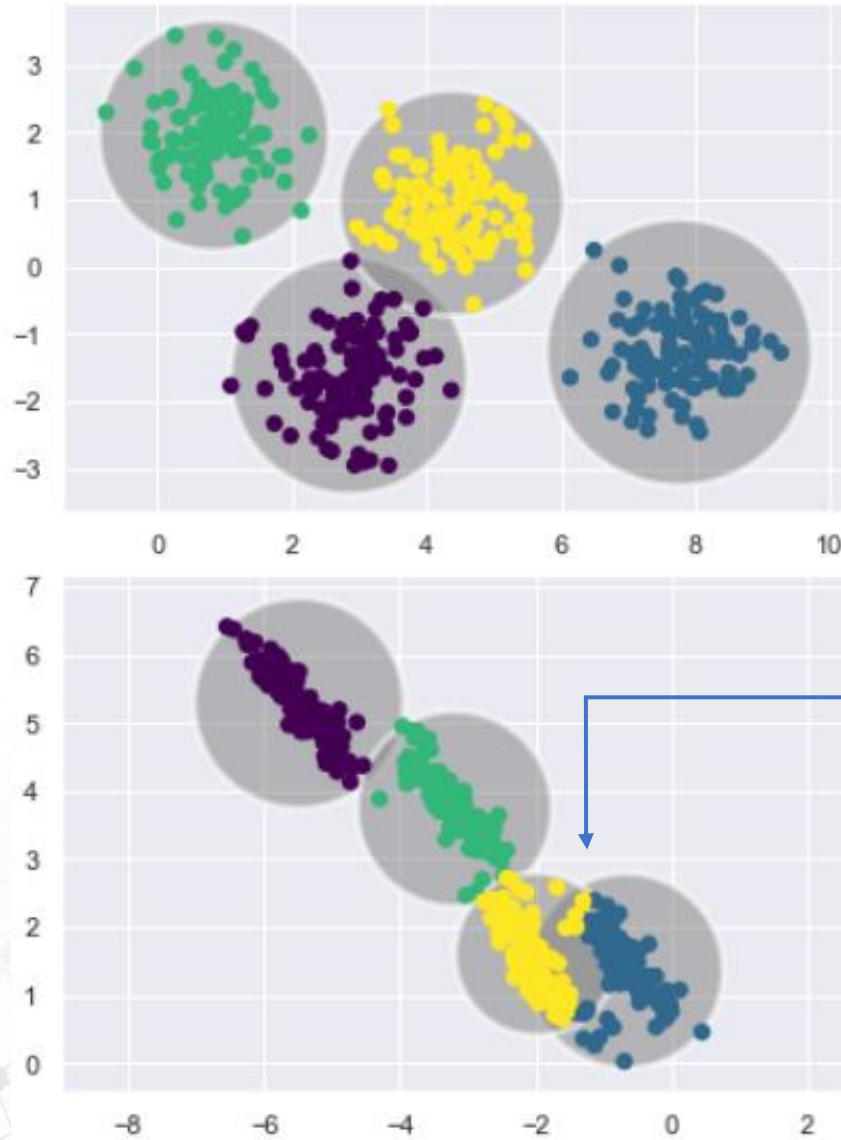
- k -means ist non-probabilistisch und basiert auf einfachen Abstandsbetrachtungen → einfach verständlich, aber fragliche Anwendbarkeit in realen Situationen
- Erinnern Sie sich an unsere typischen Bilder beim k -means Clustering: „well-behaved“ Datensätze konnte k -means gut trennen bzw. clustern
- Aber wir hatten keine Aussage darüber, wie wahrscheinlich ein Datenpunkt zu einem Cluster gehört – ob sich der Algorithmus evtl. sogar unsicher ist in der Zuweisung
- Bei DBSCAN hatten wir hierzu schon Möglichkeiten kennengelernt → Gruppierung in Core-, Density-Reachable- und Noise-Punkte – jedoch eine kontinuierliche Wahrscheinlichkeitsaussage war auch hier nicht wirklich drin
- Dieses Problem gehen wir nun an → GMM

Improving k -means



Was denken Sie?

Wie müssen wir uns die Ergebnisse eines k -means im Datenraum vorstellen – als Entscheidungsfläche?



- Lassen Sie uns das **Ergebnis** eines k -means Clusterings so vorstellen: es werden um jeden Cluster Kreise (bzw. **Hypersphären**) gespannt, die die Cluster-Zugehörigkeiten im Datenraum repräsentieren (wenn Sie die Hypersphären immer weiter vergrößern kommt es zu **linearen Trennhyperebenen**)
- Noch stärker formuliert: die Modelle der jeweiligen Cluster *müssen* sphärisch sein → wir haben damit **keine Möglichkeit** elliptische bzw. **nicht-konvexe Strukturen** zu modellieren
- Bei nicht-sphärischen Datensätzen kann es dann zu einer **Vermischung der Clusterzuordnung** kommen

0

So what?

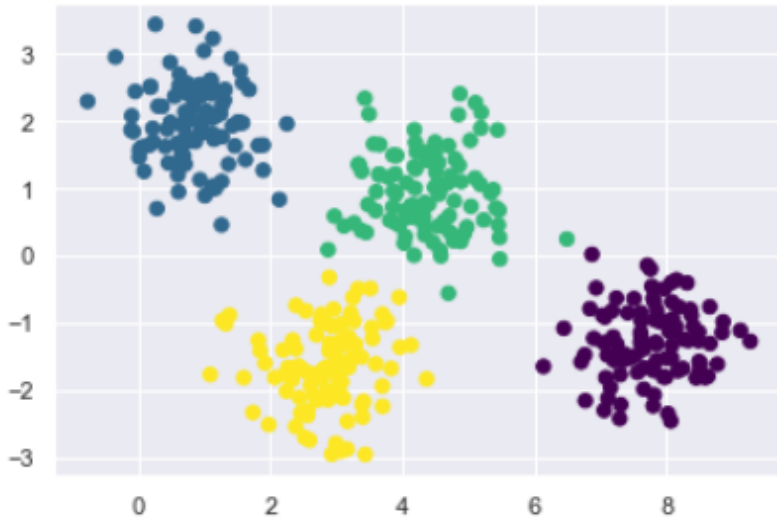
k -means hat also zwei problematische Nachteile: non-probabilistisches Verhalten und starke Annahmen über die Cluster-Strukturen

Towards GMMs



Was denken Sie?
Was könnte sich hinter
Gaussian Mixture
Models verbergen?

```
1 from sklearn.mixture import GaussianMixture
2 gmm = GaussianMixture(n_components=4).fit(X)
3 labels = gmm.predict(X)
4 plt.scatter(X[:, 0], X[:, 1], c=labels, s=40, cmap='viridis');
```



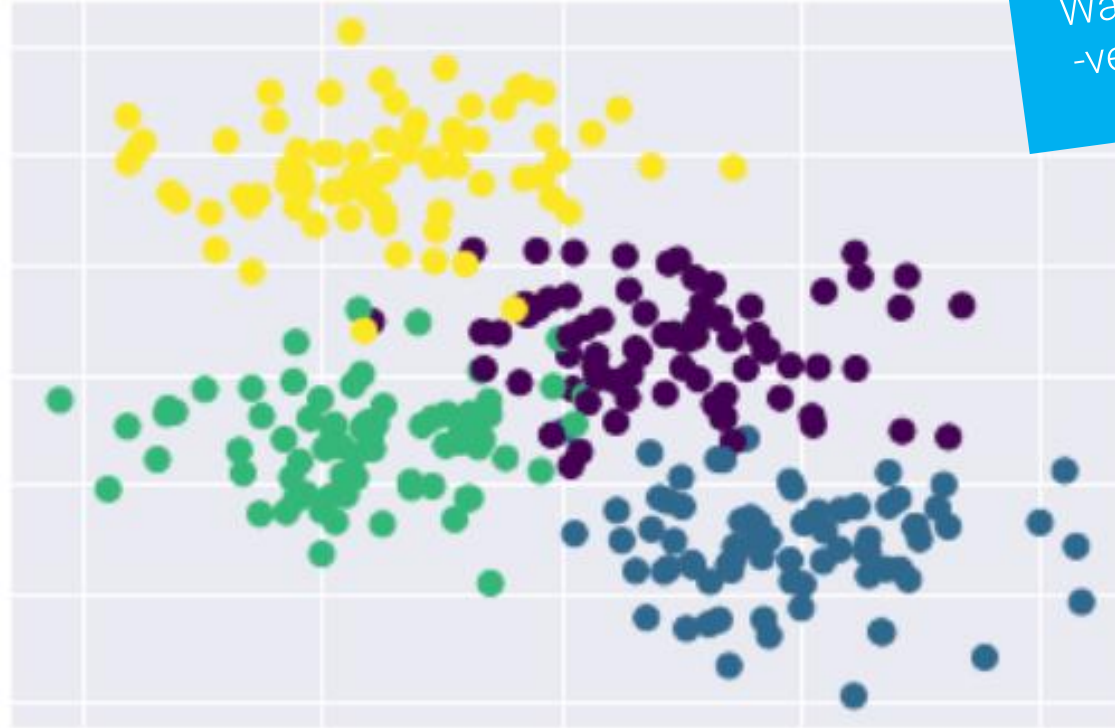
- Ein **Gaussian Mixture Model (GMM)** versucht mittels der Kombination mehrerer, **mehrdimensionaler Normalverteilungen** bzw. **Gauß'scher Wahrscheinlichkeitsdichten** beliebig strukturierte Datensätze zu modellieren
- Die zugehörige Klasse aus `sklearn` lautet
`GaussianMixture()` aus dem Modul `sklearn.mixture`
- Wir können dieses GMM erstmal auf die gleiche Weise wie einen k-means verwenden

Wir als Machine Learning Model



Was denken Sie?

Was bedeutet es die Daten mittels
Wahrscheinlichkeitsdichten bzw.
-verteilungen zu repräsentieren?
(Zoom: zeichnen)

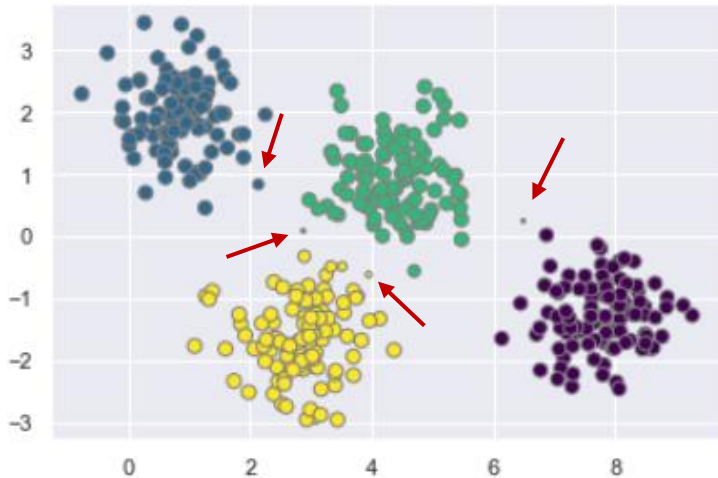


GMM und `.predict_proba()`

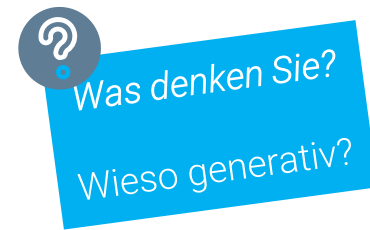
```
1 probs = gmm.predict_proba(X)
2 print(probs[:5].round(3))
```

```
[[0.463 0.    0.537 0.   ]
 [0.    0.    0.    1.   ]
 [0.    0.    0.    1.   ]
 [0.    0.    1.    0.   ]
 [0.    0.    0.    1.   ]]
```

```
1 size = 50 * probs.max(1) ** 4 # square emphasizes differences
2 plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=size,
3             edgecolors='gray');
```



- Wie wir schon bei anderen Modellen kennengelernt hatten, gibt es manchmal neben der `.predict()`- auch die `.predict_proba()`-Methode
- Da wir hier ein **probabilistisches, generatives** Modell vorliegen haben, können wir diese Methode bei GMMs nutzen, um die **Wahrscheinlichkeit der Clusterzugehörigkeit** anzugeben bzw. berechnen zu lassen – wir visualisieren uns das mal



GMM unter der Haube



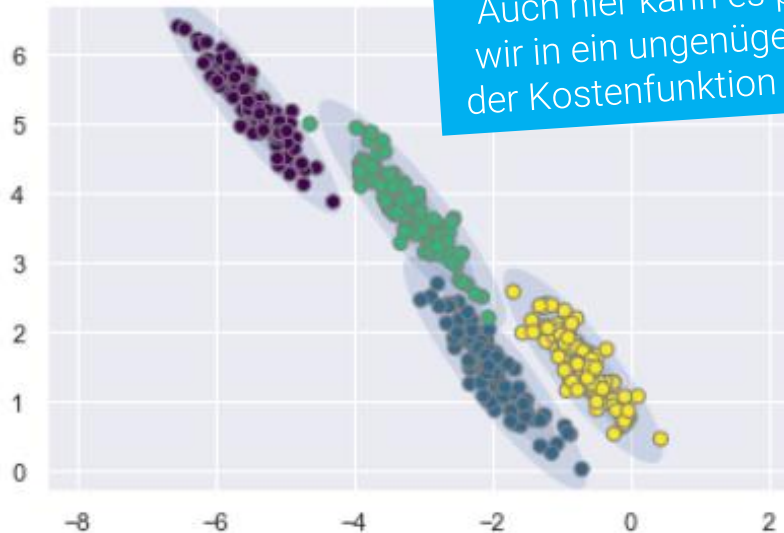
Was denken Sie?

GMMs nutzen ein grundlegendes Prinzip des k-means: welches könnte das sein?



Was denken Sie?

Auch hier kann es passieren, dass wir in ein ungenügendes Minimum der Kostenfunktion laufen: was tun?



- GMMs nutzen – wie k-means auch – den *Expectation-Maximization-Algorithmus*
- **Ablauf:**
 1. **Zufällige Initialisierung** des Ortes und der Form unserer „Gaussians“
 2. Wiederhole bis Konvergenz:
 - i. **E-Step:** berechne für jeden Datenpunkt eine Hilfsvariable, die die **Wahrscheinlichkeit der Cluster-Zugehörigkeit** des Datenpunktes repräsentiert
 - ii. **M-Step:** aktualisiere für jeden Gaussian den Ort und die Form (**Mittelwert, Kovarianz, Gewichtung**) mittels der Werte der Hilfsvariablen – Anpassung eines Parameters bei Fixierung der anderen
- Jeder Cluster wird dann also durch ein „smoothes“ **Gauß'sches Modell** repräsentiert
→ Die „**Summe**“ aller Gaussians: das **Modell** unserer Daten

GMM unter der Haube



Demo
Wir sehen uns an der Tafel
an wie zweidimensionale
Gauß-Verteilungen aussehen



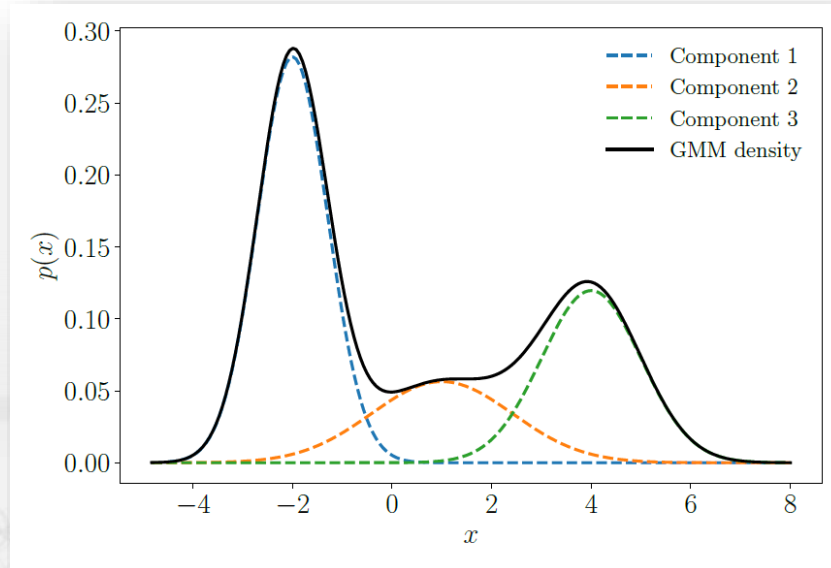
Was denken Sie?

Wie würden Sie ein *Mixture Model* generisch ausdrücken?



$$p(\mathbf{x}|\boldsymbol{\theta}) = \sum_{k=1}^K w_k p_k(\mathbf{x}|\boldsymbol{\theta})$$

$$p(\mathbf{x}|\boldsymbol{\theta}) = \sum_{k=1}^K w_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$



- Jetzt haben wir die Mechanik von GMMs kennengelernt – uns fehlt aber noch die mathematische Beschreibung
- Eine **Mischung** aus Modellen ist für uns einfach eine **Linearkombination gewichteter Wahrscheinlichkeitsdichten**
- Jetzt ersetzen wir einfach die allgemeinen Wahrscheinlichkeitsdichten durch **multivariate Normalverteilungen**
- Im **Eindimensionalen** können wir uns das anhand eines Beispiels so vorstellen:

$$p(x|\boldsymbol{\theta}) = 0.5\mathcal{N}(x|-2, 0.5) + 0.2\mathcal{N}(x|1, 2) + 0.3\mathcal{N}(x|4, 1)$$

0

So what?

Hier sehen wir schön was eine *Kombination* bzw. *Mischung* an (Gauß'schen) Modellen bedeutet

A dramatic, dark scene with a figure on a ledge looking up at a glowing orb. The figure is a small, hooded person standing on a narrow, stone-like ledge that extends from the bottom right towards the center. They are holding a small, glowing light source. Above them, a large, bright, orange-yellow orb of fire or light is suspended in the air, with a long, thin, glowing trail of light curving around it. The background is dark and textured, suggesting a cave or a deep, underground space. The overall atmosphere is mysterious and intense.

Deep Dive: Maximum Likelihood

GMM unter der Haube: Maximum Likelihood

a posteriori a priori Likelihood

$$p(\boldsymbol{\theta} \mid \mathbf{x}) = \frac{p(\boldsymbol{\theta}) p(\mathbf{x} \mid \boldsymbol{\theta})}{p(\mathbf{x})}$$

Evidence

- Erinnern Sie sich an den Satz von Bayes – $\boldsymbol{\theta}$ sind die **Parameter** unseres Modells und die $\mathbf{X}$ unsere **Features**
- An was sind wir interessiert? Daran, wie sich unsere **Daten** im **Feature-Raum verteilen** abhängig von **Parametern** eines (unbekannten) Modells → **Likelihood**
- Wir wenden einen Trick an: die Likelihood lässt sich **faktorisieren**, wenn wir annehmen, dass jeder Datenpunkt *unabhängig und identisch verteilt ist* (im naturwissenschaftlichen Sinne: aufeinanderfolgende Experimente beeinflussen sich nicht gegenseitig)

$$p(\mathbf{x} \mid \boldsymbol{\theta}) = \prod_{n=1}^N p(\mathbf{x}_n \mid \boldsymbol{\theta}) = \prod_{n=1}^N \sum_{k=1}^K w_k p_k(\mathbf{x}_n \mid \boldsymbol{\theta}) = \prod_{n=1}^N \sum_{k=1}^K w_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$



Was denken Sie?

Was kann man also mit der Likelihood machen? Wobei hilft sie uns?

0

So what?

Lassen Sie uns die *Likelihood* verbalisieren: „die Wahrscheinlichkeit dafür, dass unsere Features auftreten bzw. dass wir unsere Features so beobachten, gegeben die Parameter meines Modells bzw. meiner Verteilung.“

GMM unter der Haube: Maximum Likelihood

$$\log p(\mathbf{x}|\boldsymbol{\theta}) = \sum_{n=1}^N \log \sum_{k=1}^K w_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

$$\boldsymbol{\theta}_{\text{ML}} = \arg \max_{\boldsymbol{\theta}=\{w_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}} \sum_{n=1}^N \log \sum_{k=1}^K w_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

- Lassen Sie uns noch **logarithmieren** – dann **summieren** wir **kleine Wahrscheinlichkeiten**, anstatt dass wir sie miteinander multiplizieren
→ Dies nennt man die *Log-Likelihood*
- Was wir nun nur noch tun wollen: sie **maximieren**
- Wir suchen also die **Parameter** unseres **Modells**, die uns diesen Ausdruck **maximiert**!
→ Das nennt man *Maximum Likelihood Schätzung (MLE)*



Was denken Sie?

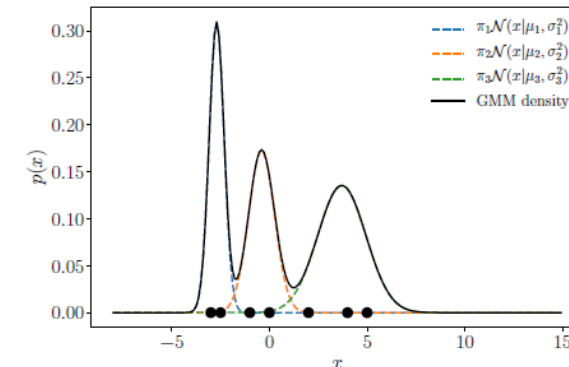
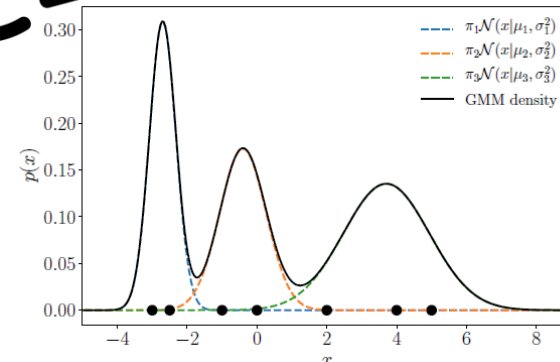
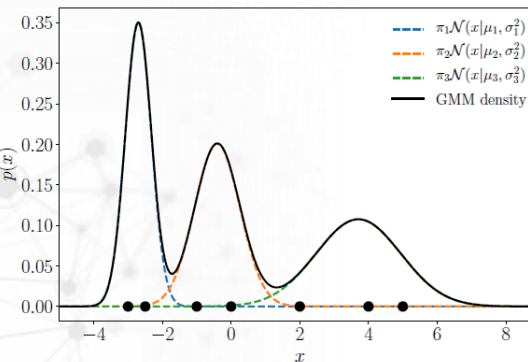
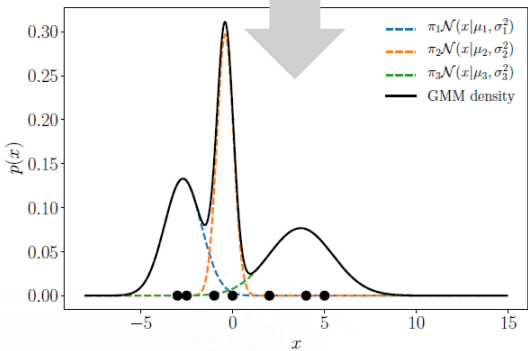
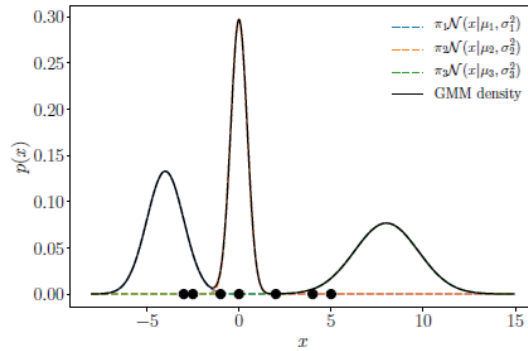
Was ist hier „ungewohnt“?



So what?

Diese Kostenfunktion – die Log-Likelihood – wird durch den E-M-Algorithmus maximiert

GMM unter der Haube: Maximum Likelihood



- Diese **Optimierung** findet nicht instantan statt, sondern es werden die Parameter *sukzessive* aktualisiert
- Sehen wir uns das in unserem einfachen Beispiel an:
 1. Nach der **Initialisierung** werden...
 2. (E) die **Clusterzugehörigkeitswahrscheinlichkeiten** berechnet
 3. (M) dann die **Mittelwerte**...,
 4. (M) dann die (Ko-)**Varianzen**
 5. (M) und letztlich die **Gewichtungen** aktualisiert
- Das kennen Sie nun von k-means bzw. dem **E-M-Algorithmus**: Schritte 2.-5. werden so oft wiederholt, bis das Modell *konvergiert*
- Ansatz harter Zuweisung zu Clustern haben wir – ähnlich wie die Werte in `.predict_proba()` – Werte, die uns angeben, wie wahrscheinlich ein Datenpunkt zu einem bestimmten Gaussian gehört
→ Diese Information wird für die **Aktualisierungsschritte** von oben genutzt

GMM unter der Haube: Aktualisierungsschritte

$$r_{nk} = \frac{w_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K w_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

$$\boldsymbol{\mu}_k^{new} = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}}$$

$$\boldsymbol{\Sigma}_k^{new} = \frac{\sum_{n=1}^N r_{nk} (\mathbf{x}_n - \boldsymbol{\mu}_k)^2}{\sum_{n=1}^N r_{nk}}$$

$$w_k^{new} = \frac{1}{N} \sum_{n=1}^N r_{nk}$$

Berechnung der Clusterzugehörigkeitswahrscheinlichkeiten

- Zu den r_{nk} sagt man **Responsibilities**
- Sie repräsentieren die **Wahrscheinlichkeit**, dass Datenpunkt $\mathbf{x}_n$ durch den **k -ten Gaussian** erzeugt wurde

Updates der Mittelwerte, Varianzen und Gewichte

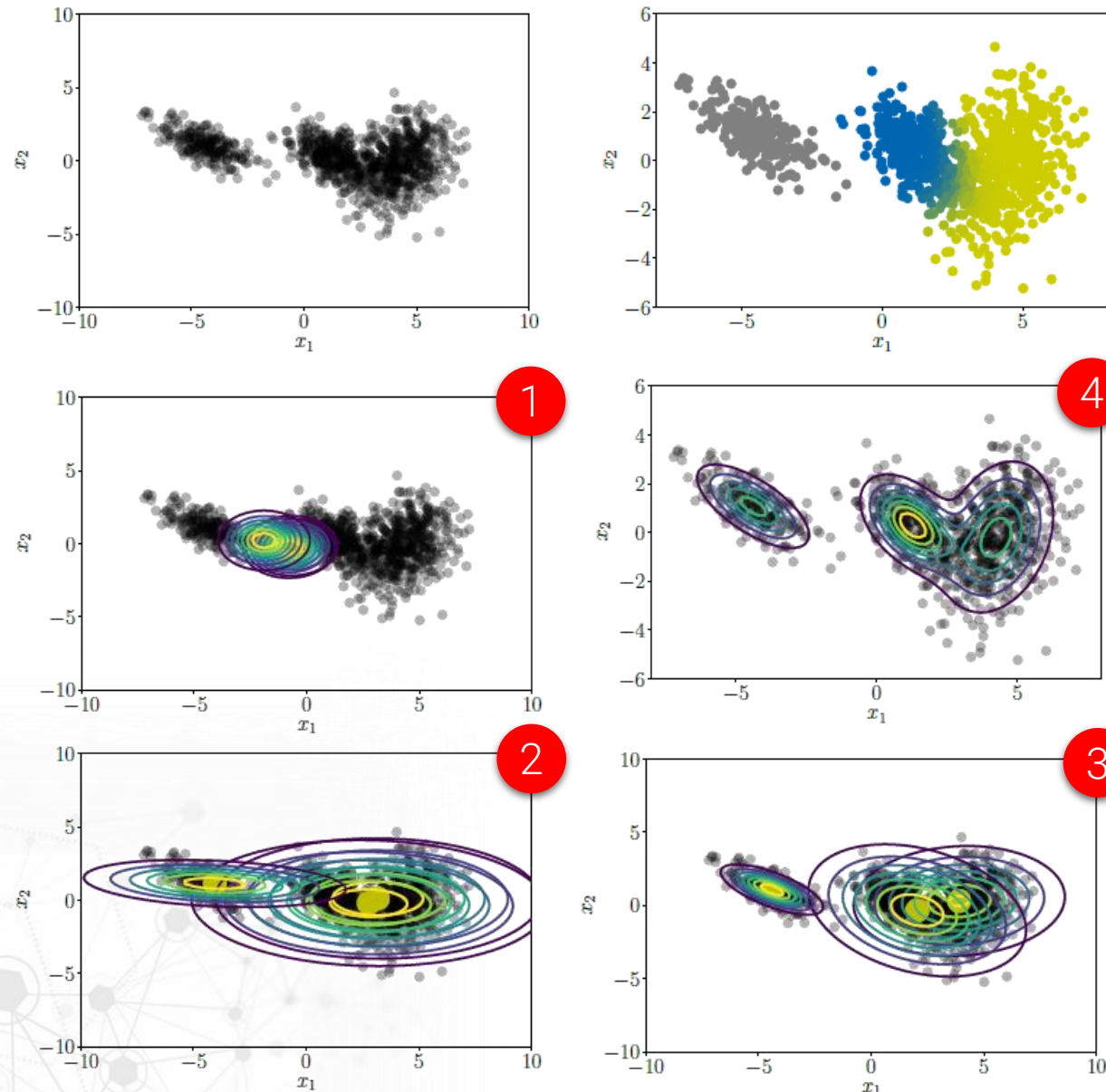
- Der neue Mittelwert ergibt sich aus der Gewichtung der Datenpunkte mit den zugehörigen Clusterzugehörigkeitswahrscheinlichkeiten → im Grunde berechnet man einen **gewichteten Mittelwert**
- Bei den anderen beiden Updates ist es nicht direkt ersichtlich, warum in der Summe die Gewichtung steht → Alle diese drei Formeln kann man herleiten, wenn man die **Log-Likelihood ableitet** und 0 setzt – wir suchen ja ein Optimum



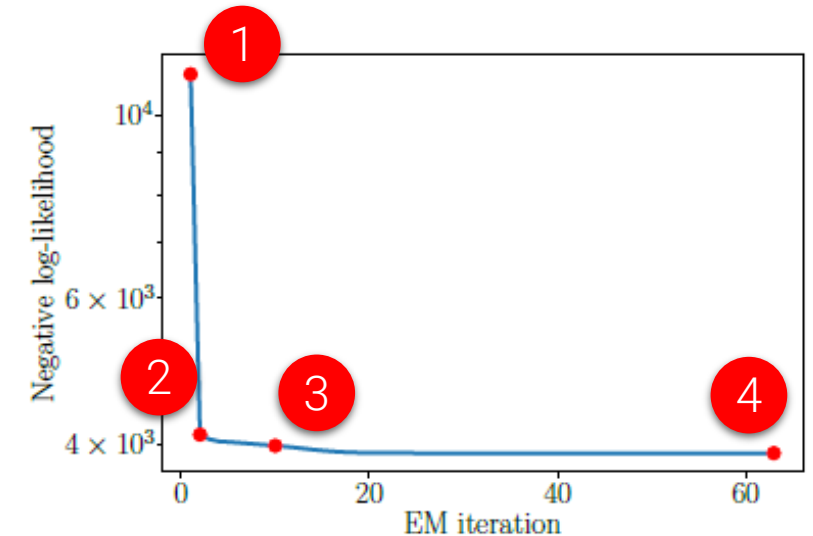
Was denken Sie?

Wie kommt man auf die Formeln?

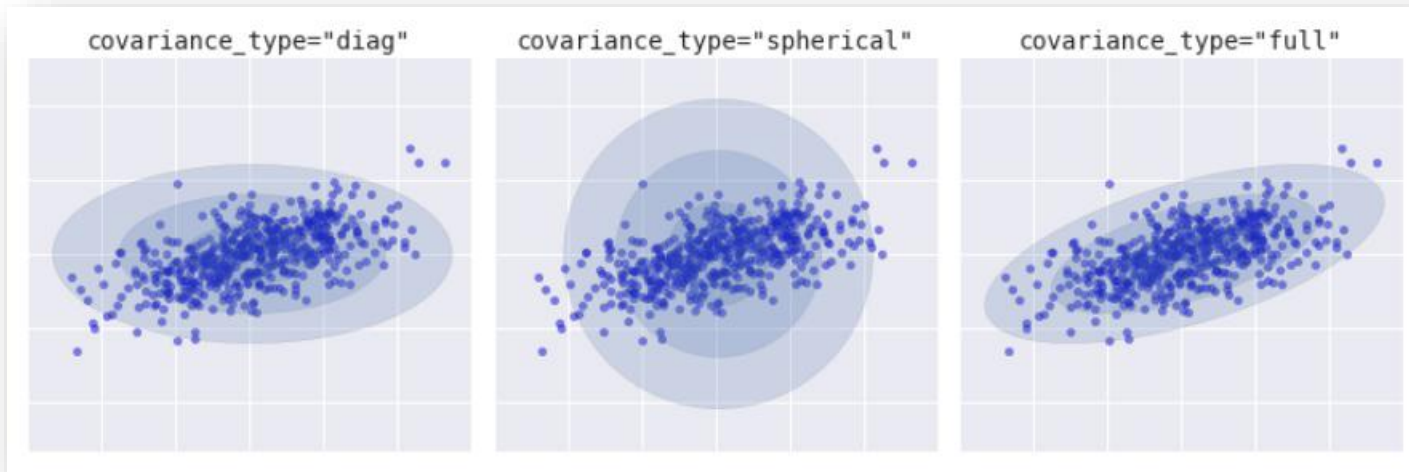
GMM unter der Haube: Maximum Likelihood



- Den Fortschritt bzw. das *Konvergenzverhalten* unserer Iterationen des E-M-Algorithmus können wir uns – ähnlich zu k-means – in einem Plot ansehen
- Wir plotten die negative Log-Likelihood und können entscheiden, wann eine genügend gute Repräsentation der Daten gefunden wurde



...zurück zu sklearn



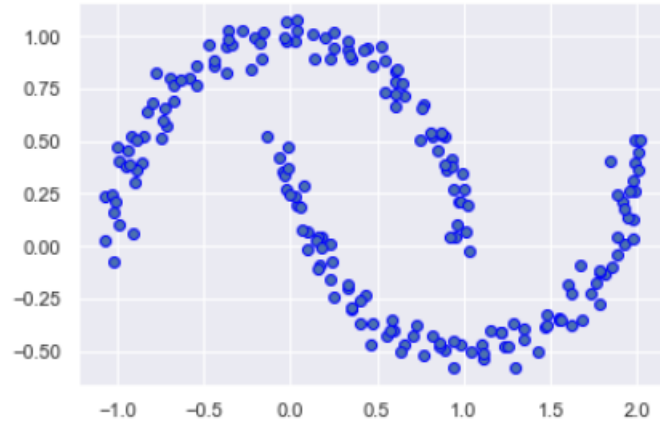
Was denken Sie?

Welche Einstellung ähnelt k-means am meisten?

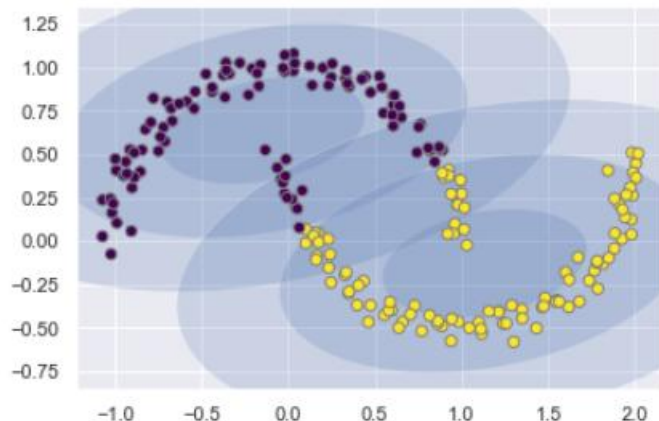
- Schauen wir uns mal einen bestimmten Hyperparameter der `GaussianMixture()`-Klasse an: den `covariance_type`
- **Möglichkeiten:**
 - `diag`: Größe des Clusters (bzw. **Streuung** des Modells) kann **für jede Dimension unabhängig** gewählt werden → Ellipse entlang der Achsen ausgerichtet
 - `spherical`: alle Dimensionen (bzw. **Streuungen** des Modells) **gleich** ausgeprägt → Ellipse = Kugel
 - `full`: alle Freiheitsgrade frei wählbar → Ellipse hat beliebige Ausrichtung

GMMs und Dichteschätzung

```
1 from sklearn.datasets import make_moons
2 Xmoon, ymoon = make_moons(200, noise=.05, random_state=0)
3 plt.scatter(Xmoon[:, 0], Xmoon[:, 1], edgecolors='blue');
```

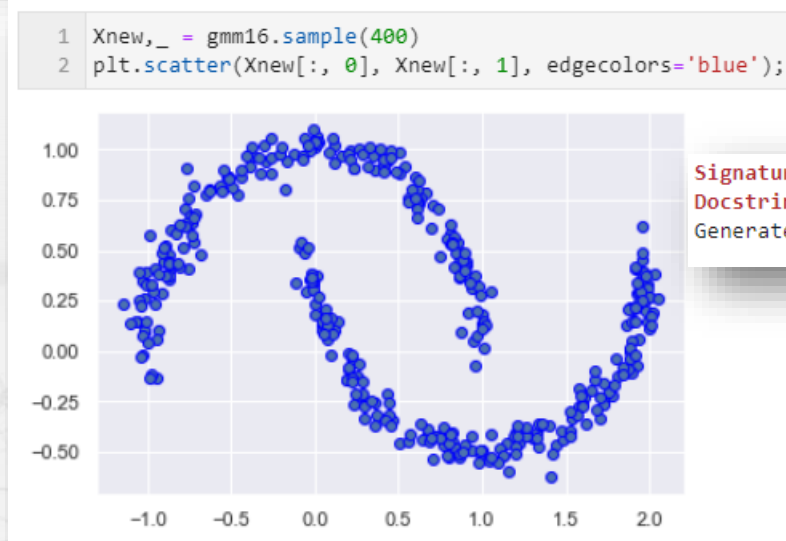


```
1 gmm2 = GaussianMixture(n_components=2, covariance_type='full', random_state=0)
2 plot_gmm(gmm2, Xmoon)
```



- GMMs werden häufig als Clustering-Algorithmen beschrieben
→ Genauer: *Dichteschätzung*
- Das Ergebnis eines GMMs ist kein Clustering-Modell, sondern ein *Generative Probabilistic Model*
- Wir sind hier also bei *generativen* Modellen – und nicht bei *diskriminativen* (wie z.B. SVMs, k-means, ...)
- Lassen Sie uns mal wieder zwei Monde malen...
- Intuitiv würden wir es evtl. mit zwei Komponenten versuchen – leider Fail...

GMMs und Dichteschätzung



Signature: `gmm16.sample(n_samples=1)`
Docstring:
Generate random samples from the fitted Gaussian distribution



Was denken Sie?

Wie machen wir das nun mit
der Anzahl der Komponenten?
Erinnert Sie das an was?



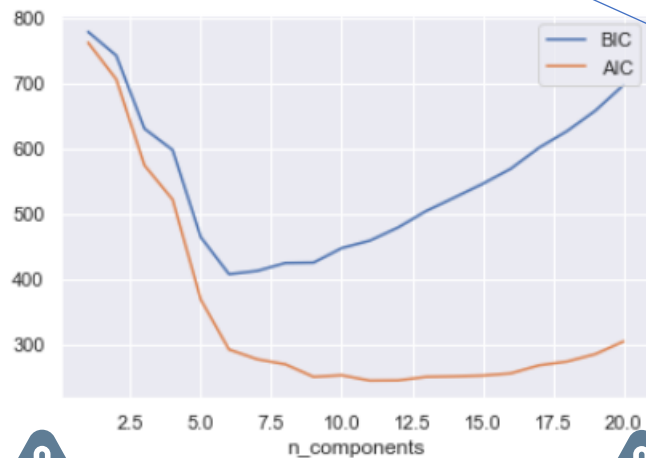
So what?

Wir haben also eine Möglichkeit, um
beliebig verteilte multidimensionale
Daten zu modellieren – und auch aus
dem Modell neue Daten zu generieren!

- Wenn wir nun aber viele (16) Komponenten zulassen – und die Cluster-Labels ignorieren – dann...
- ...erhalten wir ein Modell, das uns die *gesamte Verteilung* unserer Daten abbildet
- Wir haben nun sogar ein *generatives Modell* vorliegen, das uns Daten **erzeugen** kann!
- Hierzu können wir die Methode `.sample()` des trainierten Modells nutzen
→ dadurch **zieht** man Daten aus einem zugrundeliegenden Modell

GMMs und Dichteschätzung: Informationskriterien

```
1 n_components = np.arange(1, 21)
2 models = [GaussianMixture(n, covariance_type='full', random_state=0).fit(Xmoon)
3             for n in n_components]
4
5 plt.plot(n_components, [m.bic(Xmoon) for m in models], label='BIC')
6 plt.plot(n_components, [m.aic(Xmoon) for m in models], label='AIC')
7 plt.legend(loc='best')
8 plt.xlabel('n_components');
```



Methoden im GMM
von sklearn zur
Berechnung dieser
beiden
Informationskriterien

So what?

Hier wird nochmal deutlich: es gibt die Parameter eines Modells – und Hyperparameter. Die Parameter finden wir hier durch eine MLE – die Hyperparameter durch Betrachtung von Informationskriterien!

So what?

"Notice the important point: this choice of number of components measures how well GMM works as a density estimator, not how well it works as a clustering algorithm. I'd encourage you to think of GMM primarily as a density estimator, and use it for clustering only when warranted within simple datasets."

- Wie bei k-means auch benötigen wir ein Maß, um die besten Hyperparameter bzw. Modelle auszuwählen
- Bei generativen/probabilistischen Modellen wie dem GMM geschieht dies über sog. *Informationskriterien*

- Zwei wichtige **Informationskriterien**:

- **BIC** (Bayesian Information Criterion):

$$\text{BIC} = m \ln(n) - 2 \ln(\hat{L})$$

- **AIC** (Akaike Information Criterion):

$$\text{AIC} = 2m - 2 \ln(\hat{L})$$

- Hierbei sind m : Anzahl an geschätzten Parametern des Modells, $\hat{L}$: das Maximum der Likelihood-Funktion, n : die Anzahl an Datenpunkten

AIC führt also eine **Penalty** für die **Parameterzahl** in unserem Modell ein, um zu *regularisieren* (Overfitting zu vermeiden)
– BIC nutzt einen noch konservativeren Term

Bei beiden Kriterien liegt das **Optimum** beim **Minimum**



Was denken Sie?

Warum diese zusätzlichen
Summanden zur
Maximum Likelihood?

Beispiel: Generate Numbers



Nutzen Sie den `load_digits` Datensatz aus `sklearn.datasets`, um mittels eines *Gaussian Mixture Models* eine Funktion zu bauen, die Ihnen neue - durch das Modell generierte - "handschriftliche" Zahlen generiert. Hier nutzen wir die Eigenschaft von GMMs aus, dass sie probabilistisch bzw. generativ sind.

